LABORATORUL 2

Meniu și navigare

Construim meniul Snake și deschidem ecranul jocului.

- La final identificăm fișierele proiectului, definim un layout XML, tratăm apăsarea unui buton și urmărim lifecycle-ul în Logcat.
- Astăzi realizăm două ecrane. Tabla și mișcarea vin în laboratoarele următoare.

2 Legătura cu cursul

Cursul 2: arhitectura Android (p. 4–8), Activity și Intent (p. 10–14), View și XML (p. 16–18), lifecycle (p. 11–12 și 20–21).

- În laborator aplicăm aceste concepte într-un meniu cu navigare. Folosim Java pentru comportament și XML pentru interfață.

3 Aplicația noastră în Android

- Schemă simplificată:

Aplicația	meniul și ecranul Snake
Framework Android	API-uri precum Activity, Intent și View
ART	execută codul aplicației, scris în Java și transformat la build în DEX
Bibliotecile native	oferă funcționalități implementate în C/C++
Nucleul Linux	gestionează procese, memorie și drivere

Straturile platformei nu sunt directoarele proiectului nostru.

4 Fișierele proiectului

app	modulul aplicației
java/	clasele Java, în vizualizarea Android
res/layout/	interfețele XML
res/values/	texte, culori și alte valori
res/drawable/	imagini și forme
AndroidManifest.xml	componente și permisiuni
Fișierele Gradle	configurarea build-ului și dependențele

? **Vrem să schimbăm textul butonului și ce se întâmplă la apăsare. În ce două locuri lucrăm?**

5 Răspuns: unde lucrăm?

ÎNTREBAREA

Unde schimbăm textul butonului și comportamentul la apăsare?

RĂSPUNS

Textul: în strings.xml.
Comportamentul: în listener-ul din clasa Java.

```
// res/values/strings.xml
<string name="start_game">Start joc</string>
// res/layout/activity_main.xml
android:text="@string/start_game"
// java/.../MainActivity.java
buttonStart.setOnClickListener(...)
```

XML descrie interfața, iar Java descrie comportamentul aplicației.

6 Activity

O Activity este o componentă Android care oferă o interfață utilizatorului. Sistemul îi gestionează ciclul de viață.

- În demonstrație: MainActivity pentru meniu și GameActivity pentru joc.

```
public class MainActivity extends AppCompatActivity {
    // Comportamentul ecranului
}
```

- AppCompatActivity este clasa AndroidX folosită ca bază.

O Activity per ecran este alegerea exemplului, nu o regulă pentru orice aplicație.

7 Legătura dintre Activity și XML

Layout-ul XML descrie elementele interfeței. setContentView() încarcă interfața activității.

```
@Override
protected void onCreate(Bundle state) {
    super.onCreate(state);
    setContentView(R.layout.activity_main);
}
```

- onCreate(): inițializarea instanței. super: apelul implementării de bază. R.layout.activity_main: identificatorul layout-ului.

Fragment simplificat; în proiect păstrăm tratarea marginilor sistemului.

? **Ce se întâmplă dacă GameActivity încarcă R.layout.activity_main?**

8 Răspuns: layout-ul greșit

ÎNTREBAREA

Ce se întâmplă dacă GameActivity încarcă R.layout.activity_main?

RĂSPUNS

findViewById(R.id.gameRoot) întoarce null. Folosirea acestui null oprește aplicația cu NullPointerException.

```
// java/.../GameActivity.java, în onCreate()
setContentView(R.layout.activity_main); // greșit
findViewById(R.id.gameRoot); // null, apoi NPE
// corect: setContentView(R.layout.activity_game);
```

Layout-ul greșit nu conține id-urile pe care le caută GameActivity. Corect: R.layout.activity_game.

9 Views și dimensiuni în XML

View: un element al interfeței. ViewGroup: container pentru alte elemente.

- LinearLayout organizează; TextView afișează text; Button primește apăsări.

```
<Button
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/start_game" />
```

- match_parent folosește spațiul oferit de părinte; wrap_content adaptează dimensiunea la conținut.

- *dp pentru dimensiuni; sp pentru text, ținând cont și de preferința de scalare a utilizatorului.*

10 Resurse și identificatori

Resursele sunt texte, layout-uri și imagini accesate prin identificatori din clasa generată R.

```
<string name="start_game">Start joc</string>
```

- În layout: android:id="@+id/buttonStart" și android:text="@string/start_game".
- @+id declară ID-ul; @string referă textul; R.id.buttonStart referă ID-ul din Java.

Textele în resurse se pot reutiliza și traduce.

? **Textul direct în XML funcționează. Ce devine mai greu dacă vrem două limbi?**

11

Răspuns: textele și traducerea

ÎNTREBAREA

Textul scris direct în XML funcționează. Ce devine dificil dacă vrem și limba engleză?

RĂSPUNS

Textul scris în mai multe layout-uri se schimbă în fiecare loc.
Din strings.xml, layout-ul folosește doar numele.

```
// res/values/strings.xml (română)
<string name="start_game">Start joc</string>
// res/values-en/strings.xml (engleză)
<string name="start_game">Start game</string>
// layout-ul rămâne neschimbat:
android:text="@string/start_game"
```

Pentru o altă limbă putem adăuga resurse precum values-en/strings.xml, fără să rescriem layout-urile.

12 Evenimente și listener

- Un listener stabilește reacția la un eveniment.

```
Button buttonStart = findViewById(R.id.buttonStart);
buttonStart.setOnClickListener(view -> {
    Log.d("Lab02", "Deschidem GameActivity");
});
```

- findViewById() caută elementul în interfața încărcată. setOnClickListener() înregistrează reacția. Expresia lambda conține codul executat la apăsare.

● *Căutăm butonul după setContentView().*

? **La deschiderea meniului, fără apăsare, apare mesajul „Deschidem GameActivity”? De ce?**

13

Răspuns: când rulează listener-ul?

ÎNTREBAREA

Mesajul «Deschidem GameActivity» apare imediat după deschiderea meniului?

RĂSPUNS

Nu. setOnClickListener() doar înregistrează reacția.
Lambda rulează când apăsăm butonul.

```
// java/.../MainActivity.java, în onCreate()
buttonStart.setOnClickListener(view -> {
    Log.d("Lab02", "Deschidem GameActivity");
});  // aici doar înregistrăm reacția
// Logcat gol până apăsăm Start
```

Definirea reacției și producerea evenimentului sunt două momente diferite.

14 Navigare cu Intent

Un Intent descrie o cerere de executare a unei acțiuni. Un Intent explicit indică destinația exactă.

```
Intent intent = new Intent(
    MainActivity.this, GameActivity.class);
startActivity(intent);
```

MainActivity.this	contextul activității curente
GameActivity.class	componenta destinație
startActivity()	solicită pornirea ei. Construirea obiectului Intent nu pornește singură ecranul

15 Manifest și revenire

Declarăm activitatea internă în interiorul elementului application:

```
<activity android:name=".GameActivity"
    android:exported="false" />
```

- La butonul Înapoi folosim finish().
- În navigarea simplă a demonstrației, GameActivity este deasupra MainActivity în back stack. Închiderea jocului dezvăluie meniul existent.

MainActivity păstrează declarația de lansare generată în proiect.

? Ce se întâmplă dacă pentru Înapoi pornim încă un MainActivity, apoi repetăm traseul?

16

Răspuns: revenirea la meniu

ÎNTREBAREA

Ce se întâmplă dacă butonul Înapoi pornește de fiecare dată un MainActivity nou?

RĂSPUNS

Se acumulează instanțe în back stack. Pentru revenire închidem ecranul curent cu finish().

```
// java/.../GameActivity.java
buttonBack.setOnClickListener(view -> {
    finish(); // corect
    // startActivity(new Intent(..MainActivity..))
    // greșit: încă un meniu în back stack
});
```

Start deschide un ecran nou; finish() închide ecranul curent și îl dezvăluie pe cel anterior.

17 Ciclul de viață al unei Activity

- Android anunță tranzițiile prin callback-uri:

onCreate()	inițializare
onStart()	devine vizibilă
onResume()	intră în starea Resumed
onPause()	părăsește Resumed
onStop()	nu mai este vizibilă
onRestart()	revenirea din Stopped, înainte de onStart()
onDestroy()	distrugerea instanței, când callback-ul este executat

- *onPause() nu înseamnă distrugere. onDestroy() nu este garantat la terminarea procesului.*

18 Observarea în Logcat

- Logcat afișează mesaje din aplicație și sistem.

```
@Override
protected void onResume() {
    super.onResume();
    Log.d("Lab02", "MainActivity onResume");
}
```

- Log.d(): mesaj de depanare. Lab02: tag-ul, filtrat cu tag:Lab02.

Urmărim lansare, Start–Înapoi și Home–revenire. Logăm și onRestart().

? **La Home și revenire rapidă, dacă instanța este păstrată, ne așteptăm la onCreate() din nou?**

19

Răspuns: revenirea după Home

ÎNTREBAREA

La revenirea rapidă după Home se execută din nou onCreate()?

RĂSPUNS

De regulă, nu, dacă instanța a fost păstrată. Vedem onRestart(), onStart() și onResume().

```
// Logcat, filtru tag:Lab02 (după Home + revenire)
MainActivity onPause
MainActivity onStop
MainActivity onRestart
MainActivity onStart
MainActivity onResume
// onCreate nu apare din nou
```

Android poate recrea Activity-ul în alte situații; de aceea observăm traseul real în Logcat.

20 Erori și comportamente de urmărit

Activity nedeclarată	startActivity() poate produce ActivityNotFoundException. Identificăm excepția și linia relevantă din codul nostru
View negăsit	findViewById() poate întoarce null. Folosirea rezultatului null produce NullPointerException
Rotația	în configurația standard recreează Activity-ul. Este un comportament normal, observabil prin callback-uri